

Zephyr OS DM9051 中斷機制完整說明

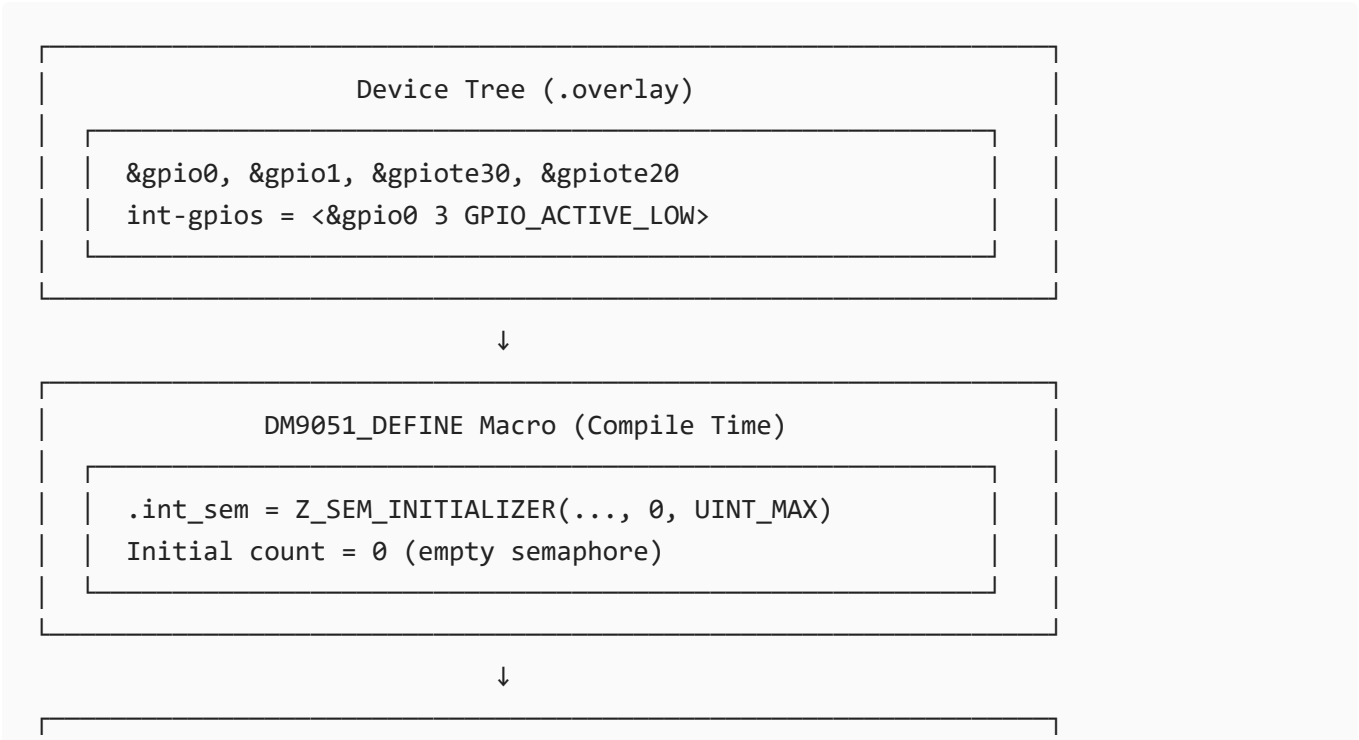
Zephyr OS DM9051 Interrupt Mechanism Complete Guide

目錄 (Table of Contents)

- 1. [系統架構概覽](#)
- 2. [Device Tree 配置層](#)
- 3. [驅動程式初始化層](#)
- 4. [信號量機制詳解](#)
- 5. [中斷處理流程](#)
- 6. [RX 執行緒運作機制](#)
- 7. [DM9051 硬體中斷控制](#)
- 8. [完整執行時序圖](#)
- 9. [實際運行日誌分析](#)

1. 系統架構概覽

1.1 整體架構圖



eth_dm9051_init() (Runtime Init)

1. Configure GPIO as input
2. gpio_init_callback(&context->gpio_cb, ...)
3. gpio_add_callback(...)
4. gpio_pin_interrupt_configure_dt(GPIO_INT_EDGE_FALLING)
5. dm9051_set_receive() → Write IMR register

↓

Runtime Interrupt Flow

```
Hardware INT → GPIOTE30 → dm9051_gpio_callback()
                → k_sem_give(&context->int_sem)
                → dm9051_rx_thread() wakes up
                → k_sem_take() returns 0
                → Process packets
                → dm9051_interrupt_reset_for_cb_sem()
```

2. Device Tree 配置層

2.1 Overlay 文件配置

文件: nrf54l15dk_nrf54l15_cpuapp.overlay

```
/* Ensure GPIO0, GPIO1 and their GPIOTE instances are enabled */
&gpio0 {
    status = "okay";      /* 啟用 GPIO Port 0 */
};

&gpio1 {
    status = "okay";      /* 啟用 GPIO Port 1 */
};

&gpiote30 {
    status = "okay";      /* 啟用 GPIOTE30 (GPIO0 的中斷控制器) */
};

&gpiote20 {
    status = "okay";      /* 啟用 GPIOTE20 (GPIO1 的中斷控制器) */
};
```

```
};

&spi21 {
    status = "okay";
    cs-gpios = <&gpio1 2 GPIO_ACTIVE_LOW>; /* CS on P1.2 */

    dm9051: dm9051@0 {
        compatible = "davicom,dm9051";
        reg = <0>;
        spi-max-frequency = <1000000>; /* 1MHz SPI 頻率 */
        int-gpios = <&gpio0 3 GPIO_ACTIVE_LOW>; /* 中斷腳位 P0.3 */
        local-mac-address = [00 11 22 33 44 55];
    };
};
```

2.2 關鍵配置說明

GPIO 與 GPIOTE 對應關係 (nRF54L15)

GPIO Port	→	GPIOTE Instance	→	用途
gpio0	→	gpiote30	→	應用核心 GPIO0 中斷
gpio1	→	gpiote20	→	應用核心 GPIO1 中斷

中斷腳位配置

```
int-gpios = <&gpio0 3 GPIO_ACTIVE_LOW>;
```

觸發極性: 低電平有效

腳位編號: Pin 3

GPIO 控制器: gpio0 (P0.3)

重要:

- 必須同時啟用 `&gpio0` 和 `&gpiote30`
- 如果只啟用 GPIO 而不啟用 GPIOTE，中斷將無法工作
- `GPIO_ACTIVE_LOW` 表示 DM9051 的 INT 腳位在有中斷時會拉低

3. 驅動程式初始化層

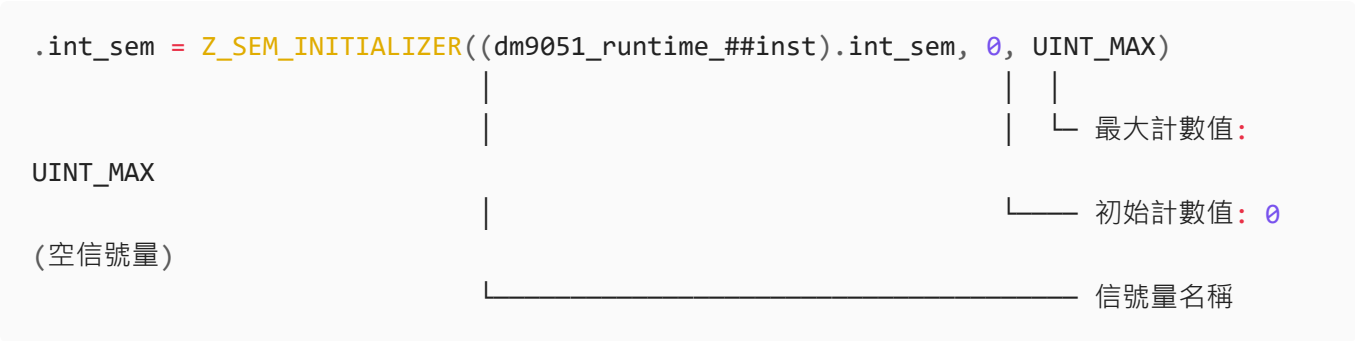
3.1 DM9051_DEFINE 宏定義

文件: eth_dm9051.c (Line 821-827)

```
#define DM9051_DEFINE(inst) \
    static struct dm9051_runtime dm9051_runtime_##inst = { \
        .mac_address = DT_INST_PROP(inst, local_mac_address), \
        .tx_rx_sem = Z_SEM_INITIALIZER((dm9051_runtime_##inst).tx_rx_sem, 1, \
UINT_MAX), \
        .int_sem = Z_SEM_INITIALIZER((dm9051_runtime_##inst).int_sem, 0, UINT_MAX), \
        .link_up = false, \
    };
```

3.2 信號量初始化詳解

Z_SEM_INITIALIZER 宏展開



關鍵特性:

- 1. 初始計數 = 0: 信號量開始時是「空的」
- 2. 最大計數 = `UINT_MAX`: 理論上可以累積無限多個信號
- 3. 編譯時初始化: 這是靜態初始化，在程式載入時就完成

兩個信號量的用途對比

信號量	初始計數	用途	保護對象
<code>.tx_rx_sem</code>	1	互斥鎖	SPI 總線訪問
<code>.int_sem</code>	0	事件通知	中斷事件同步

4. 信號量機制詳解

4.1 信號量狀態轉換

初始狀態

```
int_sem 計數器: 0  
RX Thread 狀態: WAITING (阻塞在 k_sem_take)
```

中斷發生時

```
步驟 1: DM9051 硬體產生中斷 (INT pin 拉低)  
      ↓  
步驟 2: GPIOTE30 偵測到 GPIO0 Pin3 下降沿  
      ↓  
步驟 3: Zephyr 呼叫 dm9051_gpio_callback()  
      ↓  
步驟 4: k_sem_give(&context->int_sem)  
      計數器: 0 → 1  
      ↓  
步驟 5: RX Thread 被喚醒  
      k_sem_take() 返回 0 (成功)  
      計數器: 1 → 0
```

4.2 k_sem_give 與 k_sem_take 配對

k_sem_give(&context->int_sem)

位置: dm9051_gpio_callback() (Line 551)

```
static void dm9051_gpio_callback(const struct device *dev,  
                                struct gpio_callback *cb,  
                                uint32_t pins)  
{  
    struct dm9051_runtime *context = CONTAINER_OF(cb, struct dm9051_runtime,  
gpio_cb);  
  
    printk("-----DM9051 INT! pins=0x%x-----\n", pins);  
    k_sem_give(&context->int_sem); // ← 信號量計數 +1，喚醒等待的執行緒  
}
```

作用:

1. 將 int_sem 計數器加 1
2. 如果有執行緒正在等待此信號量，立即喚醒它

3. 這是在中斷上下文中執行的，必須非常快速

k_sem_take(&context->int_sem, K_MSEC(100))

位置: dm9051_rx_thread() (Line 571)

```
static void dm9051_rx_thread(void *arg1, void *arg2, void *arg3)
{
    const struct device *dev = arg1;
    struct dm9051_runtime *context = dev->data;

    while (1) {
        if (cint(dev)) {
            /* 中斷模式：等待 GPIO 中斷信號 */
            int res = k_sem_take(&context->int_sem, K_MSEC(100));
            if (res != 0) {
                /* 超時 - 100ms 內沒有收到中斷 */
                dm9051_link_status(dev); // 檢查鏈路狀態
                continue;
            }
            /* 成功取得信號量 - 有中斷發生 */
            dm9051_interrupt_disble_irq(dev); // 暫時關閉 DM9051 中斷
        }

        /* 處理封包 */
        k_sem_take(&context->tx_rx_sem, K_FOREVER);
        while (dm9051_rx_packet(dev) == 0);
        k_sem_give(&context->tx_rx_sem);

        /* 重新啟用中斷 */
        dm9051_interrupt_reset_for_cb_sem(dev);
    }
}
```

返回值:

- 0: 成功取得信號量 (有中斷發生)
- 非0: 超時 (100ms 內沒有中斷)

5. 中斷處理流程

5.1 GPIO 中斷配置流程

位置: `eth_dm9051_init()` (Line 741-763)

```
/* 步驟 1: 檢查 GPIO 是否就緒 */
if (!gpio_is_ready_dt(&config->interrupt)) {
    LOG_ERR("GPIO port %s not ready", config->interrupt.port->name);
    return -EINVAL;
}

/* 步驟 2: 配置 GPIO 為輸入模式 */
if (gpio_pin_configure_dt(&config->interrupt, GPIO_INPUT)) {
    LOG_ERR("Unable to configure GPIO pin %u", config->interrupt.pin);
    return -EINVAL;
}

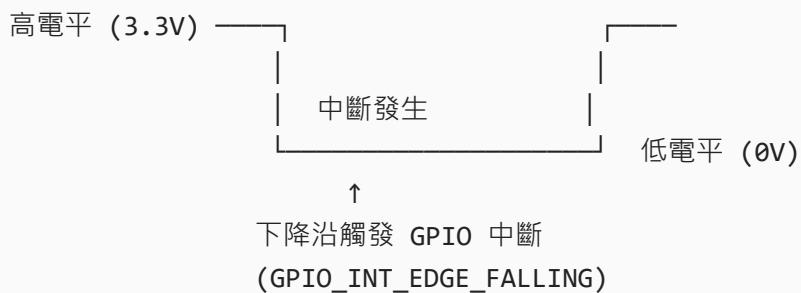
/* 步驟 3: 初始化回調函數 */
gpio_init_callback(&context->gpio_cb,           // 回調結構
                  dm9051_gpio_callback,         // 回調函數指標
                  BIT(config->interrupt.pin));   // Pin 3 → BIT(3) = 0x08

/* 步驟 4: 註冊回調到 GPIO 驅動 */
if (gpio_add_callback(config->interrupt.port, &(context->gpio_cb))) {
    return -EINVAL;
}

/* 步驟 5: 配置中斷觸發條件 */
gpio_pin_interrupt_configure_dt(&config->interrupt, GPIO_INT_EDGE_FALLING);
//                                     └─ 下降沿觸發
```

5.2 中斷觸發條件

DM9051 INT Pin 狀態:



GPIO_ACTIVE_LOW 的意義:

- Device Tree 中定義 `GPIO_ACTIVE_LOW`
- Zephyr 會自動處理極性轉換

- 驅動程式看到的邏輯: 中斷發生 = 邏輯 1

5.3 完整中斷路徑

硬體層

DM9051 芯片

- └ 內部中斷條件滿足 (封包接收、發送完成等)
 - └ IMR (Interrupt Mask Register) 允許
 - └ ISR (Interrupt Status Register) 設置位元
 - └ INT Pin 拉低 (Active Low)

↓

nRF54L15 硬體層

P0.3 (GPIO0 Port, Pin 3)

- └ 偵測到電壓下降 (3.3V → 0V)
 - └ GPIOTE30 (GPIO Task and Event)
 - └ 產生 GPIOTE 事件
 - └ 觸發 NVIC 中斷

↓

Zephyr OS 中斷處理層

NVIC Handler

- └ Zephyr GPIO 驅動
 - └ 查找註冊的回調函數
 - └ 呼叫 dm9051_gpio_callback()
 - └ pins = 0x08 (BIT(3))

↓

驅動程式層

dm9051_gpio_callback()

- └ printk("DM9051 INT! pins=0x8")
 - └ k_sem_give(&context->int_sem)
 - └ 喚醒 dm9051_rx_thread()

6. RX 執行緒運作機制

6.1 執行緒創建

位置: `eth_dm9051_iface_init()` (Line 647-651)

```
/* 創建 RX 執行緒用於封包接收 */
k_thread_create(&context->thread,                // 執行緒結構
                context->thread_stack,            // 堆疊空間
                CONFIG_ETH_DM9051_RX_THREAD_STACK_SIZE, // 堆疊大小
                dm9051_rx_thread,                // 執行緒函數
                (void *)dev, NULL, NULL,          // 參數
                K_PRIO_COOP(2),                  // 優先級：協作式優先級 2
                0,                                // 選項
                K_NO_WAIT);                       // 立即啟動
k_thread_name_set(&context->thread, "dm9051_rx");
```

6.2 RX 執行緒主循環

```
static void dm9051_rx_thread(void *arg1, void *arg2, void *arg3)
{
    const struct device *dev = arg1;
    struct dm9051_runtime *context = dev->data;

    while (1) { // ← 無限循環
        /* =====
        * 階段 1: 等待中斷或超時
        * ===== */
        if (cint(dev)) {
            /* 中斷模式 */
            int res = k_sem_take(&context->int_sem, K_MSEC(100));

            if (res != 0) {
                /* 超時路徑: 100ms 內沒有中斷 */
                dm9051_link_status(dev); // 檢查鏈路狀態
                continue; // 回到循環開始
            }

            /* 成功路徑: 收到中斷信號 */
            dm9051_interrupt_disable_irq(dev); // 暫時關閉 DM9051 中斷
        } else {
            /* 輪詢模式 */
            k_sem_take(&context->int_sem, K_MSEC(10));
        }
    }
}
```

```

/* =====
 * 階段 2: 處理封包 (臨界區)
 * ===== */
k_sem_take(&context->tx_rx_sem, K_FOREVER); // 取得 SPI 總線鎖

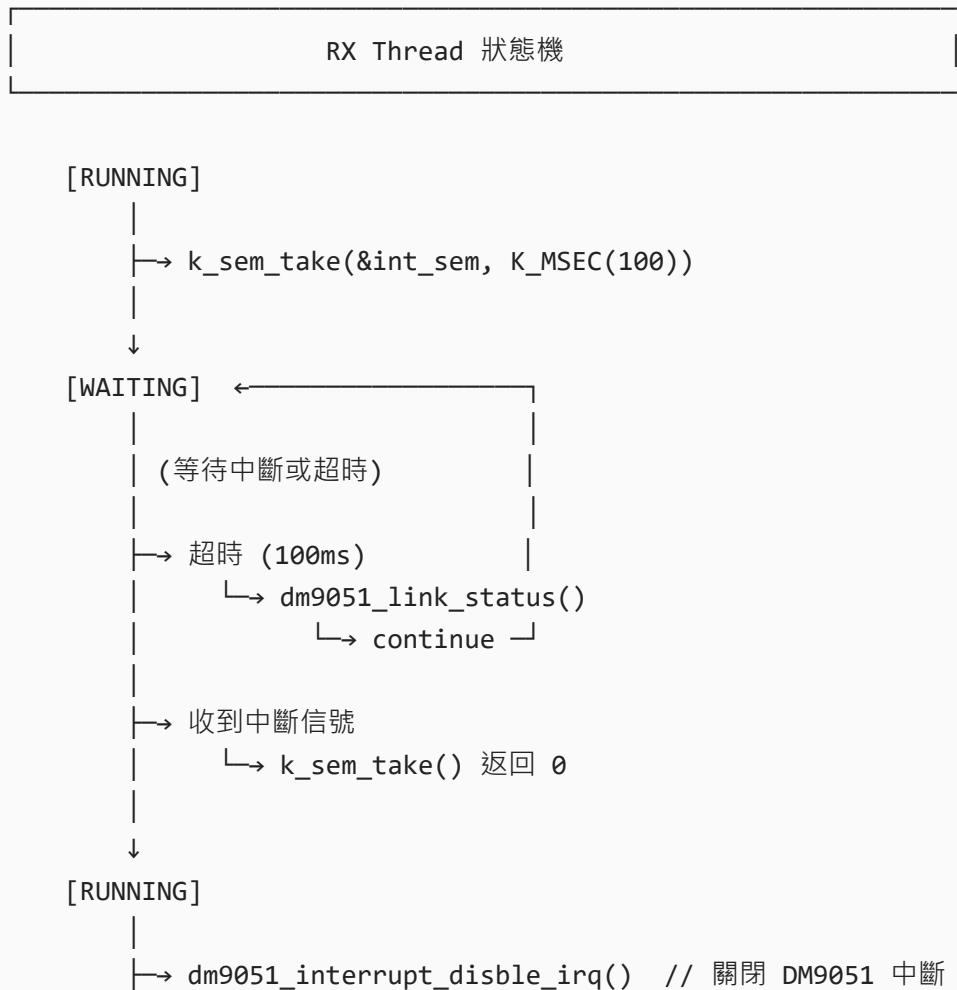
/* 處理所有可用的封包 */
while (dm9051_rx_packet(dev) == 0)
    ; // dm9051_rx_packet() 返回 0 表示還有封包

k_sem_give(&context->tx_rx_sem); // 釋放 SPI 總線鎖

/* =====
 * 階段 3: 重新啟用中斷
 * ===== */
dm9051_interrupt_reset_for_cb_sem(dev);
// └ 清除 ISR 並重新啟用 IMR
}
}

```

6.3 執行緒狀態轉換圖



```

|
|→ k_sem_take(&tx_rx_sem, K_FOREVER) // 取得 SPI 鎖
|
|→ 處理封包
|
|→ k_sem_give(&tx_rx_sem) // 釋放 SPI 鎖
|
|→ dm9051_interrupt_reset_for_cb_sem() // 重新啟用中斷
|
|→ 回到循環開始 _____

```

7. DM9051 硬體中斷控制

7.1 中斷相關暫存器

```

/* DM9051 中斷相關暫存器 */
#define DM9051_ISR    (0x7E) // Interrupt Status Register (中斷狀態暫存器)
#define DM9051_IMR    (0x7F) // Interrupt Mask Register (中斷遮罩暫存器)

/* IMR 位元定義 */
#define IMR_PAR        (1 << 7) // Pointer Auto-Return
#define IMR_PRM        (1 << 0) // Packet Received Mask
#define IMR_INT_DEFAULT (IMR_PAR | IMR_PRM) // 0x81
#define IMR_POL_DEFAULT (IMR_PAR)           // 0x80

```

7.2 中斷控制函數

初始化時啟用中斷

位置: dm9051_set_receive() (Line 343-350)

```

/* 根據 Device Tree 配置選擇中斷模式 */
if (cint(dev)) {
    /* 中斷模式：啟用封包接收中斷 */
    dm9051_write_reg(dev, DM9051_IMR, IMR_INT_DEFAULT); // 寫入 0x81
} else {
    /* 輪詢模式：只啟用 Auto-Return */
    dm9051_write_reg(dev, DM9051_IMR, IMR_POL_DEFAULT); // 寫入 0x80
}

```

暫時關閉中斷

位置: `dm9051_interrupt_disble_irq()` (Line 210-213)

```
void dm9051_interrupt_disble_irq(const struct device *dev)
{
    dm9051_write_reg(dev, DM9051_IMR, IMR_PAR); // 寫入 0x80
    // └ 只保留 Auto-Return · 關閉封包接收中斷
}
```

呼叫時機: 在 RX 執行緒收到中斷信號後立即呼叫 (Line 578)

目的: 防止在處理封包期間產生新的中斷

重新啟用中斷

位置: `dm9051_interrupt_reset_for_cb_sem()` (Line 225-229)

```
static void dm9051_interrupt_reset_for_cb_sem(const struct device *dev)
{
    dm9051_isr_enab(dev); // 清除 ISR 狀態位元
    dm9051_imr_enab(dev); // 重新啟用 IMR
}

void dm9051_isr_enab(const struct device *dev)
{
    uint8_t isrs = dm9051_read_reg(dev, DM9051_ISR);
    dm9051_write_reg(dev, DM9051_ISR, isrs); // 寫回以清除狀態
}

void dm9051_imr_enab(const struct device *dev)
{
    dm9051_write_reg(dev, DM9051_IMR, IMR_INT_DEFAULT); // 寫入 0x81
}
```

呼叫時機: 處理完所有封包後 (Line 593)

目的: 允許 DM9051 再次產生中斷

7.3 中斷控制時序

時間軸 →

[1] 初始化

```
dm9051_set_receive()
└→ IMR = 0x81 (啟用中斷)
```

[2] DM9051 接收到封包

- ↳ ISR 位元被設置
- ↳ INT Pin 拉低

[3] GPIO 中斷觸發

```
dm9051_gpio_callback()  
↳ k_sem_give(&int_sem)
```

[4] RX 執行緒被喚醒

```
k_sem_take() 返回 0  
↳ dm9051_interrupt_disble_irq()  
    ↳ IMR = 0x80 (關閉封包中斷)
```

[5] 處理封包

```
k_sem_take(&tx_rx_sem)  
↳ dm9051_rx_packet() × N  
↳ k_sem_give(&tx_rx_sem)
```

[6] 重新啟用中斷

```
dm9051_interrupt_reset_for_cb_sem()  
↳ 讀取並寫回 ISR (清除狀態)  
↳ IMR = 0x81 (重新啟用中斷)
```

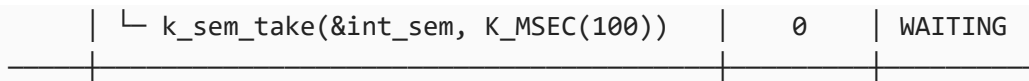
[7] 回到步驟 2 (等待下一個封包)

8. 完整執行時序圖

8.1 從開機到第一個中斷

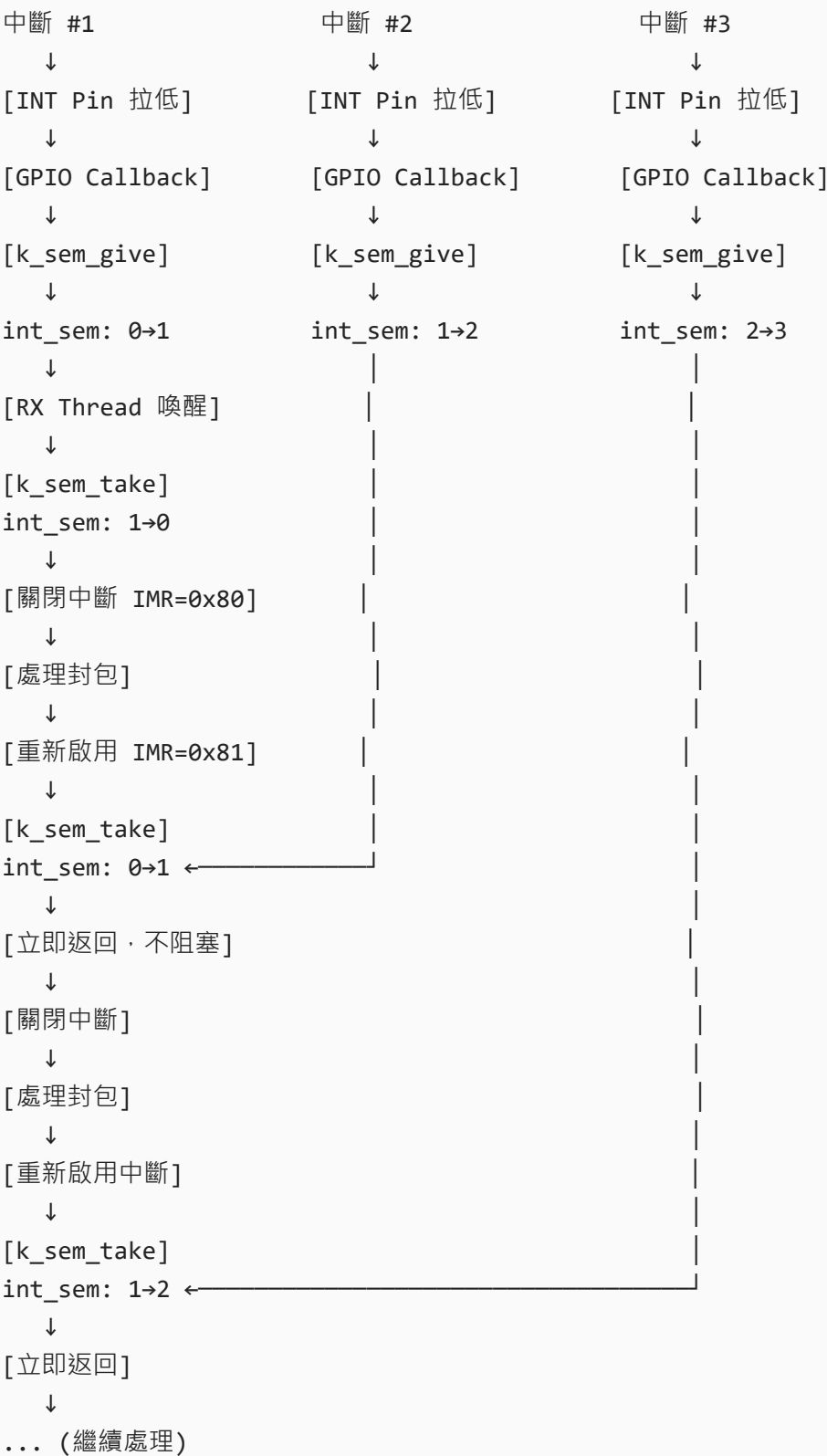
時間	事件	int_sem	執行緒狀態
T0	系統開機	0	N/A
	DM9051_DEFINE 宏展開		
	↳ int_sem 靜態初始化 (count=0)		
T1	eth_dm9051_init() 開始	0	N/A
	↳ 配置 GPIO 為輸入		
	↳ gpio_init_callback()		
	↳ gpio_add_callback()		
	↳ gpio_pin_interrupt_configure_dt()		
	↳ dm9051_set_receive() ↳ IMR = 0x81 (啟用中斷)		
T2	eth_dm9051_iface_init()	0	N/A

	└ k_thread_create(&dm9051_rx_thread) └ 執行緒立即啟動		CREATED
T3	dm9051_rx_thread() 開始執行 └ k_sem_take(&int_sem, K_MSEC(100)) └ 信號量為 0，執行緒阻塞	0	RUNNING
		0	WAITING
T4	(等待中斷...) 100ms 超時 └ k_sem_take() 返回 -EAGAIN └ dm9051_link_status() └ 檢測到鏈路連接 └ printf("Link up") └ continue (回到循環)	0	WAITING
		0	RUNNING
		0	RUNNING
T5	再次 k_sem_take(&int_sem, K_MSEC(100))	0	WAITING
T6	DM9051 接收到第一個封包 └ ISR 位元被設置 └ INT Pin 拉低 (3.3V → 0V)	0	WAITING
T7	GPIOTE30 偵測到下降沿 └ 觸發 NVIC 中斷 └ Zephyr GPIO 驅動處理 └ dm9051_gpio_callback() └ printf("INT! 0x8") └ k_sem_give(&int_sem)	0	WAITING
		0→1	WAITING
T8	RX 執行緒被喚醒 └ k_sem_take() 立即返回 0 └ dm9051_interrupt_disable_irq() └ IMR = 0x80 (關閉中斷)	1	READY
		1→0	RUNNING
		0	RUNNING
T9	k_sem_take(&tx_rx_sem, K_FOREVER) └ 取得 SPI 總線鎖 └ dm9051_rx_packet() └ 讀取並處理封包 └ net_recv_data() 送到網路堆疊	0	RUNNING
T10	k_sem_give(&tx_rx_sem) └ 釋放 SPI 總線鎖 - dm9051_interrupt_reset_for_cb_sem() └ 讀取並清除 ISR └ IMR = 0x81 (重新啟用中斷)	0	RUNNING
T11	回到循環開始	0	RUNNING



8.2 連續中斷處理

當 DM9051 連續接收多個封包時：



關鍵觀察:

- 信號量可以累積計數 (0→1→2→3...)
- 如果中斷來得很快，RX 執行緒可以連續處理而不阻塞
- 這就是為什麼日誌中會看到連續多個 "DM9051 INT! pins=0x8"

9. 實際運行日誌分析

9.1 日誌解讀

```
_eth_dm9051_init: Configuring INTERRUPT mode
_eth_dm9051_init: Interrupt GPIO configured - Port: gpio@10a000, Pin: 3
```

說明: GPIO0 (基址 0x10a000) Pin 3 已配置為中斷輸入

```
(end.e=0) BUILD_VERSION Configuring INTERRUPT mode
dm9051_init.e: (set mac address, 00:11:22:33:44:55) Chip ID: 0x9051
```

說明: DM9051 初始化完成，芯片 ID 正確

```
(link_status.o=3)
[00:00:01.785,564] <inf> eth_dm9051: dm9051@0: Link up
```

說明:

- 在 T4 時刻 (超時後)
- RX 執行緒呼叫 dm9051_link_status()
- 檢測到網路鏈路已連接

```
-----DM9051 INT! pins=0x8-----
-----DM9051 INT! pins=0x8-----
-----DM9051 INT! pins=0x8-----
```

說明:

- pins=0x8 = BIT(3) = GPIO Pin 3
- 連續多個中斷表示 DM9051 快速接收多個封包
- 每次中斷都會呼叫 dm9051_gpio_callback()
- 信號量計數累積: 0→1→2→3...


```
(handler.e=4)
[00:00:09.128,346] <inf> net_dhcpv4_client_sample: Address[1]: 192.168.6.20
[00:00:09.128,369] <inf> net_dhcpv4_client_sample: Subnet[1]: 255.255.255.0
[00:00:09.128,395] <inf> net_dhcpv4_client_sample: Router[1]: 192.168.6.1
```

說明: DHCP 成功獲取 IP 地址，證明中斷驅動的封包接收正常工作

9.2 日誌重複打印現象

您提到的「重複打印現象」是正常的，原因:

1. Zephyr Console 緩衝機制

- UART 輸出可能會有緩衝
- 高速打印時可能出現重複

2. 多核心系統

- nRF54L15 可能有多個核心
- 不同核心的日誌可能交錯

3. 中斷上下文打印

- `dm9051_gpio_callback()` 在中斷上下文中打印
- 可能與主執行緒的打印交錯

建議: 這不影響功能，只是視覺上的重複

10. 總結

10.1 關鍵要點

1. Device Tree 配置

- 必須同時啟用 GPIO 和對應的 GPIOTE
- `int-gpios` 定義中斷腳位和極性

2. 信號量機制

- `.int_sem` 初始化為 0 (空信號量)
- `k_sem_give()` 在中斷回調中執行 (計數 +1)
- `k_sem_take()` 在 RX 執行緒中等待 (計數 -1)
- 可以累積計數，支援連續中斷

3. 中斷控制

- 收到中斷後立即關閉 DM9051 中斷 (IMR)
- 處理完封包後重新啟用中斷

- 防止中斷風暴

4. 執行緒同步

- `int_sem`: 中斷事件通知
- `tx_rx_sem`: SPI 總線互斥保護

10.2 優勢

- ✓ 低延遲: 中斷驅動，封包到達立即處理
- ✓ 低功耗: 不需要持續輪詢
- ✓ 高效率: CPU 可以處理其他任務
- ✓ 可靠性: 信號量機制保證不丟失中斷事件

10.3 注意事項

- ⚠ **GPIO/GPIOTE 配對:** 必須正確啟用
- ⚠ **中斷極性:** 確認 `GPIO_ACTIVE_LOW` 與硬體匹配
- ⚠ **SPI 總線保護:** 使用 `tx_rx_sem` 避免競爭
- ⚠ **中斷重入:** 在處理期間關閉 DM9051 中斷

附錄: 相關代碼位置索引

功能	文件	行號
Device Tree 配置	<code>nrf54l15dk_nrf54l15_cpuapp.overlay</code>	23-67
DM9051_DEFINE 宏	<code>eth_dm9051.c</code>	821-827
GPIO 中斷配置	<code>eth_dm9051.c</code>	741-763
GPIO 回調函數	<code>eth_dm9051.c</code>	543-552
RX 執行緒主循環	<code>eth_dm9051.c</code>	560-595
中斷關閉函數	<code>eth_dm9051.c</code>	210-213
中斷重啟函數	<code>eth_dm9051.c</code>	225-229
初始化中斷啟用	<code>eth_dm9051.c</code>	343-350

文件版本: 1.0

創建日期: 2025-12-05

作者: Antigravity AI Assistant

適用於: Zephyr OS v4.1.99, nRF Connect SDK v3.1.0, DM9051 Driver